

PROJECT DOCUMENTATION

AnswerBot – AI-Powered FAQ and Customer Support Automation

1.INTRODUCTION

- **Team ID:** SWTID-2026-7674

- **Team Leader:**

Name: Divya R

Reg No: 222406496

NM ID: DEA895D348A8C95AE0D4A59A8B2EB034

- **Team Members:**

1. Name: Mageshwari S

Reg No: 222406513

NM ID: E531CA0E63D99A4B69E45877CB2F0EF1

2. Name: Padmaja B

Reg No: 222406519

NM ID: FBE44E514924556A4994F441D472C3FF

3. Name: Rabyathul Basarya M

Reg No: 222406522

NM ID: 65399C808887238986C480070509C975

2. ABSTRACT

AnswerBot is an AI-powered FAQ and customer support automation system designed to help users find answers to frequently asked questions and help content creators prepare and manage FAQ records. The backend is built using Node.js and Express.js, with MongoDB and Mongoose for data storage and validation. JWT-based authentication and bcrypt password hashing are used to protect user accounts and restricted operations. Google Gemini AI integration is used to generate FAQ question-and-answer content. The system also supports FAQ categorization, search, and CRUD operations through RESTful APIs.

3. PROJECT OVERVIEW

Customer support teams often receive repeated questions and spend considerable time preparing, updating, and organizing answers. AnswerBot provides a centralized backend to manage FAQ content and support AI-assisted content generation. Public users can view and search published FAQ content, while authenticated creators can create and manage their own records.

4. PROBLEM STATEMENT

- Preparing and maintaining FAQ documentation manually is time-consuming.
- Repeated customer questions increase the workload of support teams.
- Organizing questions by topic and category can be difficult.
- Users may have difficulty finding relevant answers through basic browsing.
- Private FAQ records require access controls to prevent unauthorized modification.

3. OBJECTIVES

- Provide a centralized platform for FAQ creation, storage, and retrieval.
- Generate FAQ questions and answers with Google Gemini AI assistance.
- Allow keyword-based FAQ search and category-wise organization.
- Protect user accounts and private operations using authentication and authorization.
- Maintain data integrity through Mongoose schema validation and centralized error handling

4. SCOPE OF THE PROJECT

The project focuses on backend REST API functionality. It includes user registration and login, FAQ CRUD operations, category assignment, search, AI-assisted FAQ generation, and role-based access concepts. API requests and responses can be tested using Postman or Thunder Client. A separate frontend interface is not specified in the supplied project reference.

5. EXISTING SYSTEM AND PROPOSED SYSTEM

5.1 Existing System

In a manual FAQ workflow, support staff draft answers, categorize content, update documents, and search for relevant information with limited automation. This can increase response preparation time and make content maintenance difficult.

5.2 Proposed System

AnswerBot centralizes FAQ records in MongoDB and exposes REST endpoints through Express.js. Authenticated users can manage their own FAQ content, while public users can view and search published information. AI assistance supports the drafting of FAQ pairs, which can then be reviewed and saved.

6. TECHNOLOGIES AND REQUIREMENTS

SOFTWARE REQUIREMENTS

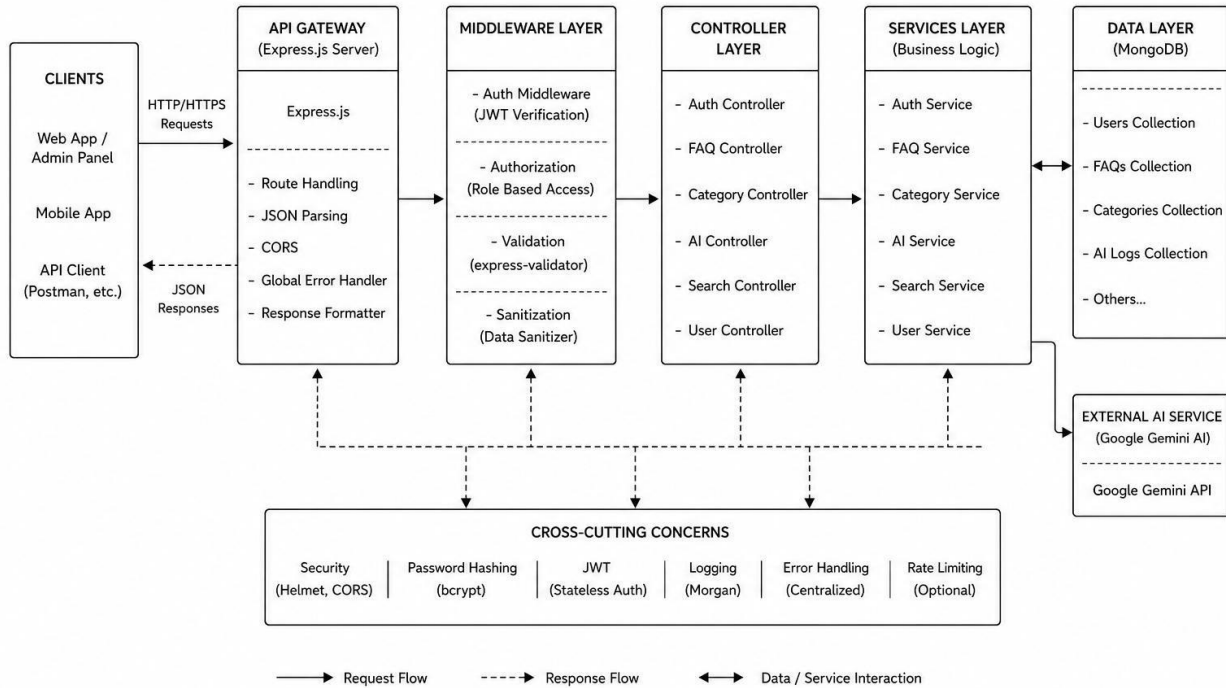
- Operating System: Windows 10/11, macOS, or Linux (supports cross-platform operations).
- Node.js (v18 or above): Runtime ecosystem running server logic and routing infrastructure.
- npm (v9 or above): Node package manager to control operational dependencies.
- Express.js (v4/v5 Ecosystem): Lightweight routing web framework to construct backend RESTful entry points.
- MongoDB / Mongoose ODM: Document NoSQL engine storing application users and curated data entities.
- Postman / ThunderClient: API verification toolkit to assert schema validation responses across administrative route shields.
- Code Editor: Visual Studio Code

HARDWARE REQUIREMENTS

- Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.
- RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB processing layers).
- Storage: 1 GB of available disk workspace.

7. SYSTEM ARCHITECTURE

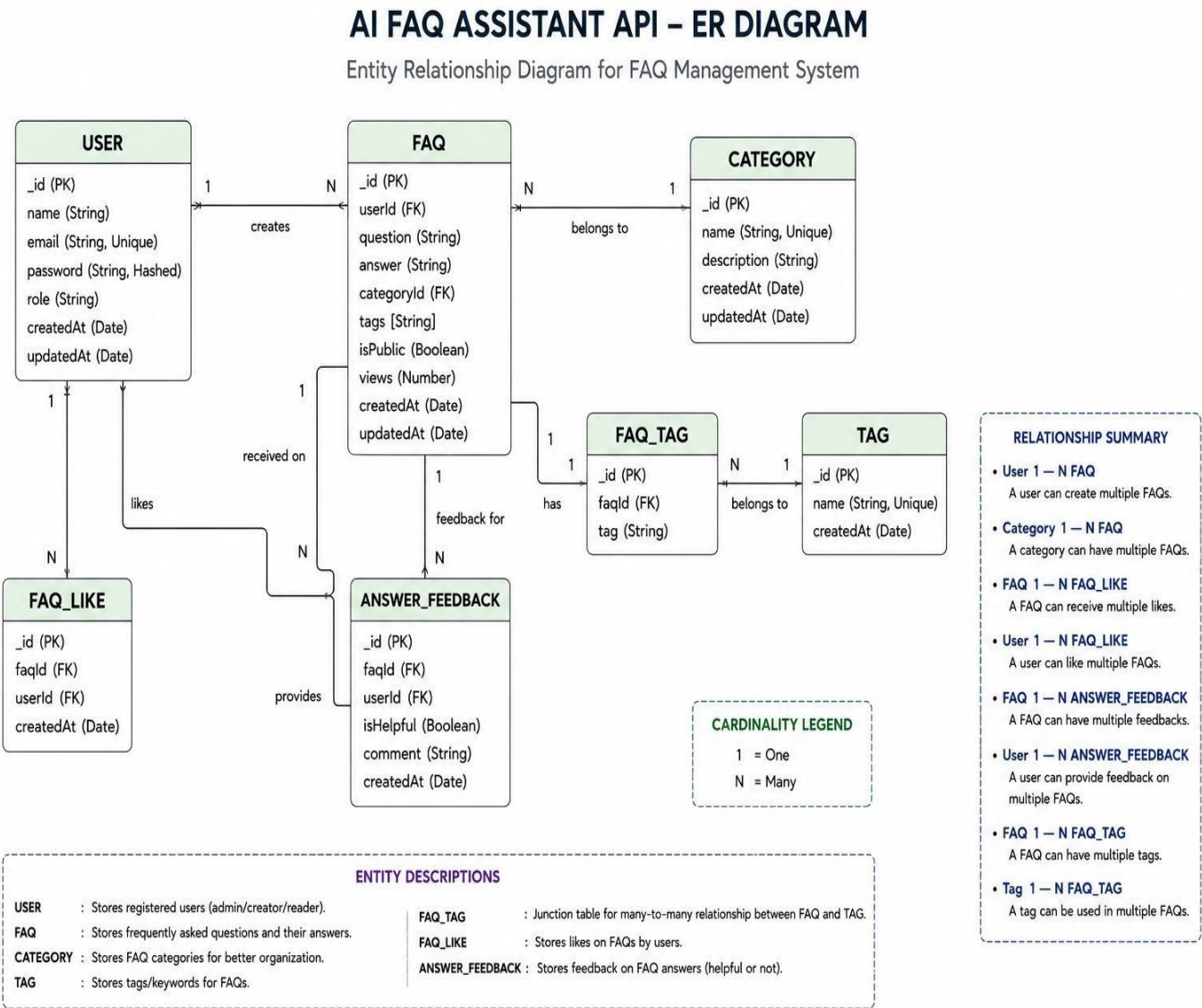
AI FAQ Assistant API – Technical Architecture



Components Definition:

- API interface: Postman or Thunder Client sends HTTP requests and receives JSON responses.
- Express server: Accepts requests, maps GET/POST/PUT/DELETE routes, parses JSON, and configures CORS.
- Authentication middleware: Reads and verifies JWT tokens for protected endpoints.
- Routes and controllers: Direct requests to application logic and prepare responses.
- Models and database layer: Mongoose schemas validate and interact with MongoDB collections.
- AI service integration: Google Gemini SDK supports FAQ generation and AI-assisted answer workflows.

8. ENTITY-RELATIONSHIP (ER) DIAGRAM DESCRIPTION



The backend database consists of multiple MongoDB collections connected through logical references to maintain user authentication, FAQ management, AI-generated content, and category organization.

1. User Entity

- `_id` (Primary Key - ObjectId)
- `name` (String, Required)
- `email` (String, Required, Unique)
- `password` (String, Hashed, Required)
- `role` (String, Default: "user")
- `createdAt`
- `updatedAt`

2. FAQ Entity

- `_id` (Primary Key - ObjectId)
- `question` (String, Required)
- `answer` (String, Required)
- `category` (String, Required)
- `createdBy` (Foreign Key referencing User Collection)
- `createdAt`
- `updatedAt`

3. AI Generation Entity (Logical)

(Generated dynamically using Google Gemini AI before persistence.)

- `topic` (String)
- `generatedQuestion` (String)
- `generatedAnswer` (String)
- `generatedCategory` (String)
- `generatedAt`

4. Category Entity (Logical)

- `categoryName` (String)
- `description` (String)
- `totalFAQs` (Number)

9.KEY FEATURES

JWT Authentication & Authorization: Secure token-based authentication protects private API endpoints.

AI FAQ Generation: Google Gemini AI automatically generates questions, answers, and topic suggestions.

Semantic FAQ Search: Intelligent keyword-based search enables faster information retrieval.

Complete CRUD Operations: Create, Read, Update, and Delete FAQ records.

Category Management: Organizes FAQs into structured categories for easier navigation.

Password Encryption: User passwords are securely hashed using bcrypt before storage.

Centralized Error Handling: Standardized API error responses improve reliability and debugging.

Schema Validation: Mongoose validation ensures data integrity before database persistence.

RESTful API Architecture: Modular Express.js routes provide scalable backend services.

MongoDB Integration: Efficient NoSQL document storage using Mongoose ODM.

Key Structural Mappings

User → FAQ (One-to-Many):

A registered user can create and manage multiple FAQ records.

Category → FAQ (One-to-Many):

Each category can contain multiple FAQ documents.

User → AI Generated FAQ (One-to-Many):

A user can generate multiple AI-assisted FAQs, which may later be saved into the FAQ collection.

FAQ → Search Results (Logical Relationship):

Search queries retrieve matching FAQ documents using keyword and semantic matching.

10. ROLES AND RESPONSIBILITIES

Admin

- Manage all registered users.
- View, update, and delete any FAQ.
- Monitor AI-generated content.
- Manage FAQ categories.
- Access complete system functionality

Content Creator

- Create new FAQs.
- Edit personal FAQ records.
- Delete owned FAQs.
- Generate FAQs using AI.
- Organize FAQs under categories.

Authenticated User

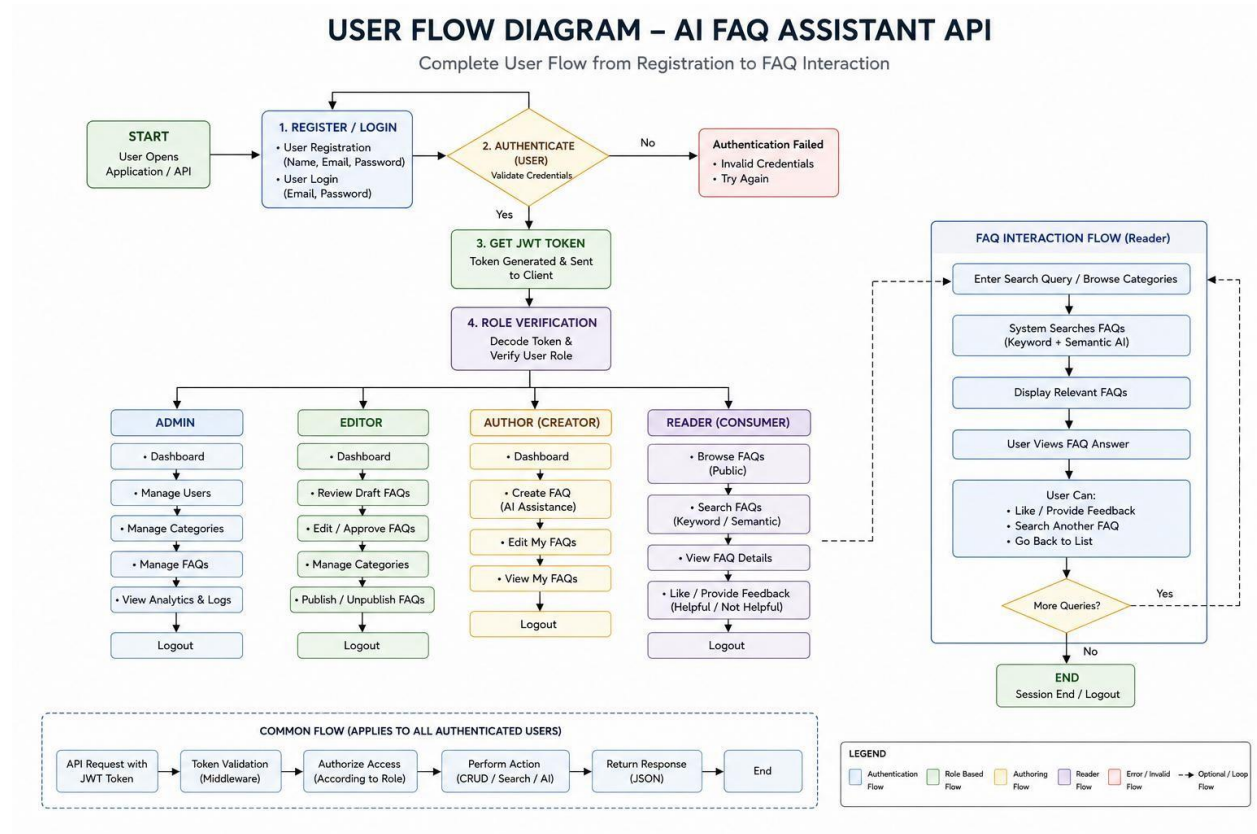
- View public FAQs.
- Search FAQs using keywords.
- Generate AI-assisted answers.
- Access personal profile information.

Public User • View

published FAQs.

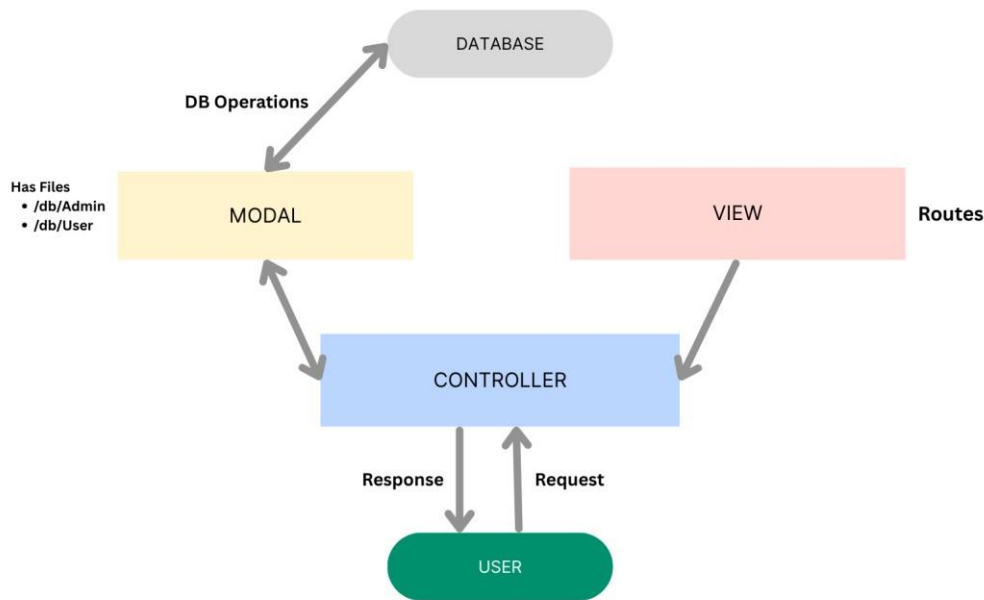
- Search FAQs.
- Cannot create, edit, or delete FAQ records.
- No access to protected API endpoints

11. USER FLOW



MVC Architecture Pattern

The system architecture structures its source configuration layers around the Model-View-Controller design pattern to completely decouple components:



Model Layer (Data Blueprint)

Defines structural blueprints via strongly enforced Mongoose schemas mapping explicit fields into raw collection items inside MongoDB. It sets field rules and defaults before validation persistence occurs.

Controller Layer (Execution Matrix)

The intermediary brain of the ecosystem. It captures request vectors from execution routes, checks structural request constraints, delegates processing workflows directly to models or services, and packages resultant output into standard JSON return arrays.

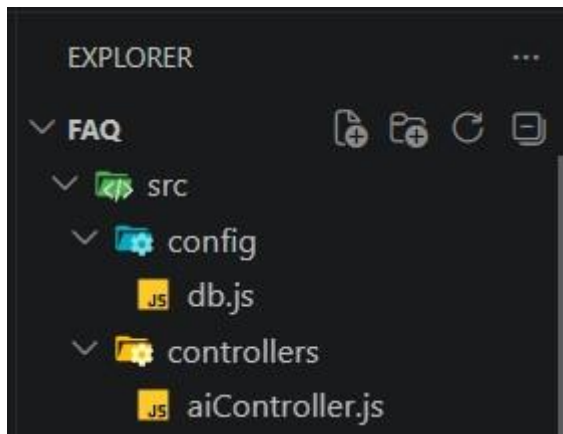
View Layer (Routing Network Interface)

In this pure headless backend service context, standard user-interface template views do not exist. Instead, the interface layer lives as an API Routing Layer. It links network requests on specified paths directly to their associated controller handler logic maps.

12. PROJECT SETUP AND CONFIGURATION

Creating Project Folder

1. Create a new folder with your FAQ.
2. Inside that folder, create a new subfolder named src.
3. Now open that folder in VS Code.

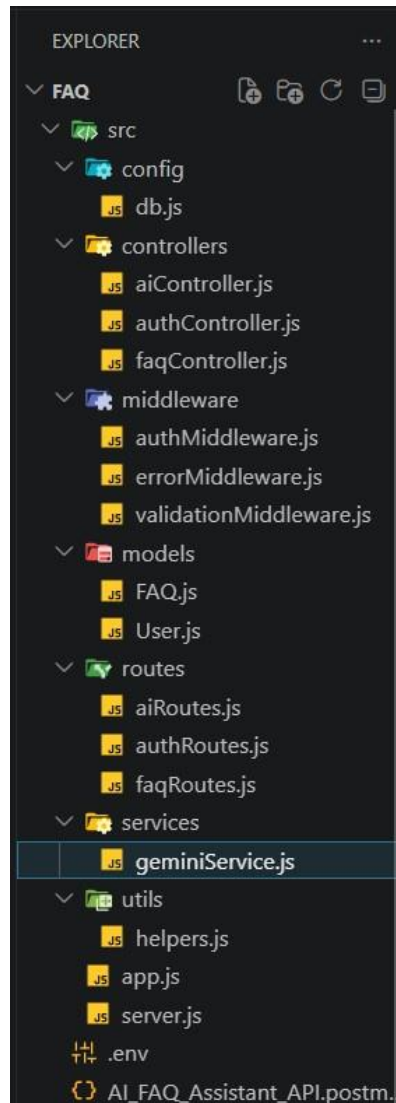


Server Setup (npm init)

Open your root folder terminal in VS Code and run: `Bash npm init -y npm`
install `express mongoose dotenv cors jsonwebtoken bcrypt @google/genai`

File and Folder Structure Matrix

Plaintext



Backend Development

Core System Script Implementations

1. Gateway Server Node (src/server.js)

This main entrance mechanism aggregates environment variables, triggers database listeners, and binds network listeners to active environment ports:

```
c > server.js > ...
1  const app = require('./app');
2  const connectDB = async () => {
3    try {
4      const dbConnect = require('./config/db');
5      await dbConnect();
6    } catch (error) {
7      console.error('Failed to connect to the database', error);
8    }
9  };
10
11 const startServer = async () => {
12   // Connect to database
13   await connectDB();
14
15   const PORT = process.env.PORT || 5000;
16
17   const server = app.listen(PORT, () => {
18     console.log(`Server running in ${process.env.NODE_ENV || 'development'} mode on port ${PORT}`);
19   });
20
21   // Handle unhandled promise rejections
22   process.on('unhandledRejection', (err, promise) => {
23     console.error(`Unhandled Rejection: ${err.message}`);
24     // Close server & exit process
25     server.close(() => process.exit(1));
26   });
27 };
28
29 startServer();
30
```

2. Gateway Express Setup (src/app.js)

```
src > app.js > ...
1  const express = require('express');
2  const cors = require('cors');
3  const dotenv = require('dotenv');
4  const { errorHandler } = require('./middleware/errorMiddleware');
5
6  // Load environment variables
7  dotenv.config();
8
9  const app = express();
10
11 // Enable CORS
12 app.use(cors());
13
14 // Body parser middlewares
15 app.use(express.json());
16 app.use(express.urlencoded({ extended: false }));
17
18 // Basic status check route
19 app.get('/', (req, res) => {
20   res.json({
21     success: true,
22     message: 'AI FAQ Assistant API is running.',
23     version: '1.0.0',
24   });
25 });
26
27 // Register API Routes
28 app.use('/api/auth', require('./routes/authRoutes'));
29 app.use('/api/faqs', require('./routes/faqRoutes'));
30 app.use('/api/ai', require('./routes/aiRoutes'));
31
32 // Handle Socket.IO polling requests gracefully (if client attempts WebSocket connection)
33 app.use('/socket.io', (req, res) => {
34   res.status(404).json({ error: 'Socket.IO not enabled' });
35 });
36
```

3. Security Access Middleware (src/middleware/authMiddleware.js)

Intercepts incoming target queries, extracts transmission header authorization records, and validates token identity context claims:

```

> middleware > src/middleware/authMiddleware.js > ...
1 const jwt = require('jsonwebtoken');
2 const User = require('../models/User');
3
4 const protect = async (req, res, next) => {
5   let token;
6
7   if (
8     req.headers.authorization &&
9     req.headers.authorization.startsWith('Bearer')
10  ) {
11    try {
12      // Get token from header
13      token = req.headers.authorization.split(' ')[1];
14
15      // Verify token
16      const decoded = jwt.verify(token, process.env.JWT_SECRET || 'your_jwt_secret_key_here_must_be_long_and_secure');
17
18      // Get user from the token, exclude password
19      req.user = await User.findById(decoded.id).select('-password');
20
21      if (!req.user) {
22        return res.status(401).json({
23          success: false,
24          message: 'Not authorized, user not found',
25        });
26      }
27
28      next();
29    } catch (error) {
30      console.error(error);
31      return res.status(401).json({
32        success: false,
33        message: 'Not authorized, token failed',
34      });
35    }
36  }
37 }

```

4. Centralized Error Handler Middleware (src/middleware/errorMiddleware.js)

```

> middleware > src/middleware/errorMiddleware.js > ...
1 const errorHandler = (err, req, res, next) => {
2   let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
3   let message = err.message;
4
5   // Respect status code set on the error object (e.g., 429 from Gemini SDK or 400/403/404)
6   if (err.status) {
7     statusCode = err.status;
8   }
9
10  // Mongoose duplicate key error (e.g. duplicate email)
11  if (err.code === 11000) {
12    statusCode = 400;
13    const field = Object.keys(err.keyValue)[0];
14    message = `Duplicate field value entered: ${field}. Please use another value.`;
15  }
16
17  // Mongoose validation error
18  if (err.name === 'ValidationError') {
19    statusCode = 400;
20    message = Object.values(err.errors)
21      .map((val) => val.message)
22      .join(', ');
23  }
24
25  // Mongoose bad ObjectId
26  if (err.name === 'CastError') {
27    statusCode = 404;
28    message = `Resource not found with id of ${err.value}`;
29  }
30
31  // Only log stack trace to the console for unexpected 500 Server Errors
32  if (statusCode === 500) {
33    console.error(err.stack);
34  } else {

```

Database Configuration

Mongoose Schema Definitions

1. Identity Resource Document Schema (src/models/User.js)

Enforces profile field criteria validations and hooks password hashing steps prior to user document persistence:

```
.env  geminiService.js  User.js  ALFAQ_Assistant_API.postman_collection.json
src > models > User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcrypt');
3
4  const userSchema = new mongoose.Schema(
5  {
6    name: {
7      type: String,
8      required: [true, 'Name is required'],
9      trim: true,
10   },
11   email: {
12     type: String,
13     required: [true, 'Email is required'],
14     unique: true,
15     lowercase: true,
16     trim: true,
17     match: [/^\S+@\S+\.\S+$/, 'Please provide a valid email address'],
18   },
19   password: {
20     type: String,
21     required: [true, 'Password is required'],
22     minlength: [6, 'Password must be at least 6 characters long'],
23   },
24 },
25 {
26   timestamps: true,
27 }
28 );
29
30 // Pre-save hook to hash the password before saving
31 userSchema.pre('save', async function () {
32   if (!this.isModified('password')) {
33     return;
34   }
35   const salt = await bcrypt.genSalt(10);
36   this.password = await bcrypt.hash(this.password, salt);
37 });
```

2. FAQ Resource Document Schema (src/models/FAQ.js)

Models core properties and maps a distinct relational reference link back to the source creator document:

```
.env  geminiService.js  FAQ.js  ALFAQ_Assistant_API.postman_collection.json  BLACKBOX
src > models > FAQ.js > ...
1  const mongoose = require('mongoose');
2
3  const faqSchema = new mongoose.Schema(
4  {
5    question: {
6      type: String,
7      required: [true, 'Question is required'],
8      trim: true,
9    },
10   answer: {
11     type: String,
12     required: [true, 'Answer is required'],
13     trim: true,
14   },
15   category: {
16     type: String,
17     required: [true, 'Category is required'],
18     enum: {
19       values: ['Technology', 'Education', 'Health', 'Banking', 'General'],
20       message: '{VALUE} is not a valid category. Choose from Technology, Education, Health, Banking, General',
21     },
22     default: 'General',
23   },
24   createdBy: {
25     type: mongoose.Schema.Types.ObjectId,
26     ref: 'User',
27     required: [true, 'Creator user reference is required'],
28   },
29 },
30 {
31   timestamps: true,
32 }
33 );
34
35 const FAQ = mongoose.model('FAQ', faqSchema);
36
```

API Router Interface Endpoints

1. Public Authentication Operations (src/routes/authRoutes.js)

```
src > routes > js authRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    registerUser,
5    loginUser,
6    getUserProfile,
7  } = require('../controllers/authController');
8  const { protect } = require('../middleware/authMiddleware');
9  const {
10    validateRegister,
11    validateLogin,
12  } = require('../middleware/validationMiddleware');
13
14  // Public routes
15  router.post('/register', validateRegister, registerUser);
16  router.post('/login', validateLogin, loginUser);
17
18  // Protected routes
19  router.get('/profile', protect, getUserProfile);
20
21  module.exports = router;
22
```

2. Protected FAQ Operations (src/routes/faqRoutes.js) JavaScript

```
src > routes > js faqRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    createFAQ,
5    getAllFAQs,
6    getFAQById,
7    updateFAQ,
8    deleteFAQ,
9    searchFAQs,
10  } = require('../controllers/faqController');
11  const { protect } = require('../middleware/authMiddleware');
12  const { validateFAQ } = require('../middleware/validationMiddleware');
13
14  // Public routes - GET operations
15  router.get('/search', searchFAQs); // Note: defined before /:id to prevent routing issues
16  router.get('/', getAllFAQs);
17  router.get('/:id', getFAQById);
18
19  // Protected routes - Modification operations
20  router.post('/', protect, validateFAQ, createFAQ);
21  router.put('/:id', protect, updateFAQ);
22  router.delete('/:id', protect, deleteFAQ);
23
24  module.exports = router;
25
```

3. AI Automation Operations (src/routes/aiRoutes.js)

```
rc > routes >  aiRoutes.js > ...
1  const express = require('express');
2  const router = express.Router();
3  const {
4    generateAIAnswer,
5    generateAIFaq,
6  } = require('../controllers/aiController');
7  const { protect } = require('../middleware/authMiddleware');
8  const {
9    validateAIAnswer,
10   validateAIFaq,
11 } = require('../middleware/validationMiddleware');
12
13 // Protected routes to prevent API overuse/abuse
14 router.post('/answer', protect, validateAIAnswer, generateAIAnswer);
15 router.post('/generate-faq', protect, validateAIFaq, generateAIFaq);
16
17 module.exports = router;
18
```

13. PROJECT EXECUTIONS & SCREEN SHOTS

Running the API Application

- Install Postman or ThunderClient (VS Code extension).
- Start the backend development server environment parameters:

Bash npm run dev

API Testing Specification Demo

To verify operational structural integrity via execution endpoints, map endpoints using the configurations below:

1. Identity Onboarding Path Registration

- Route: POST /api/auth/register
- Access: Public
- Payload Request Body JSON:

JSON

{

```

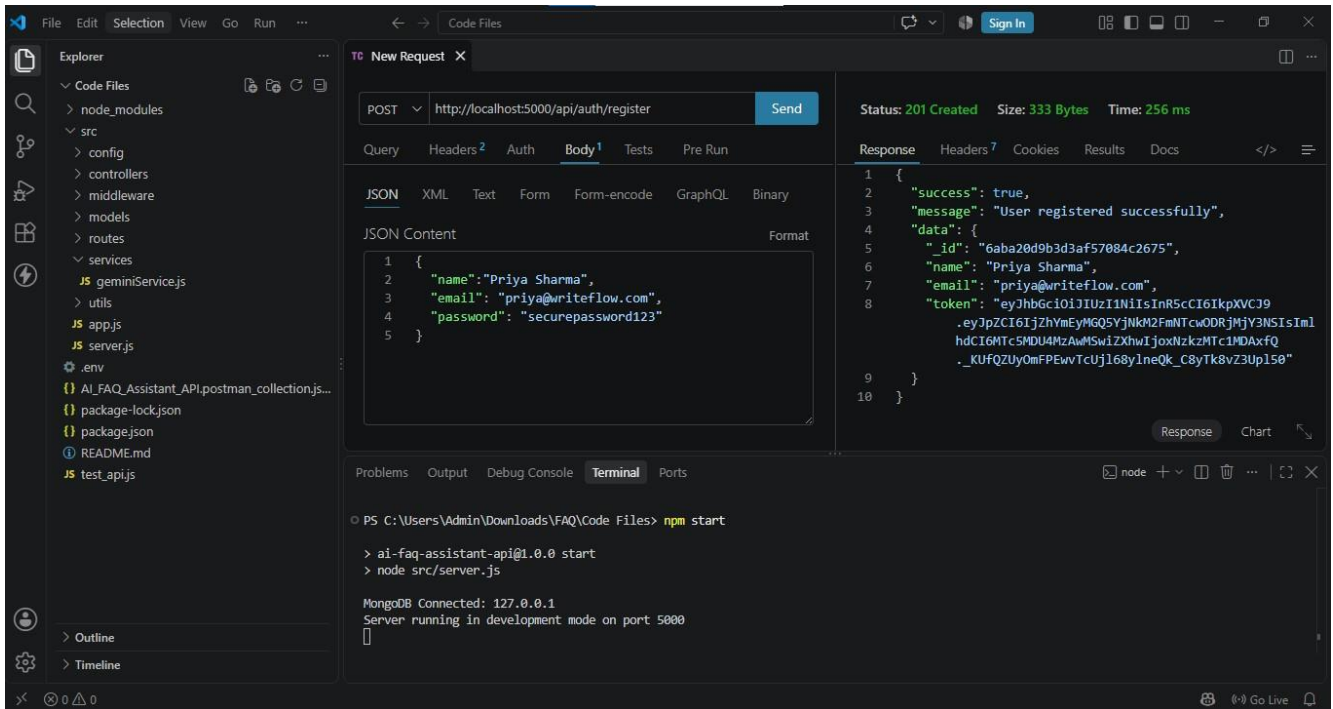
"name": "Priya Sharma",

"email": "priya@writeflow.com",

"password": "securepassword123"

}

```



2. Profile Login Verification Quey

- Route: POST `/api/auth/login`
- Access: Public
- Payload Request Body JSON:

JSON

```

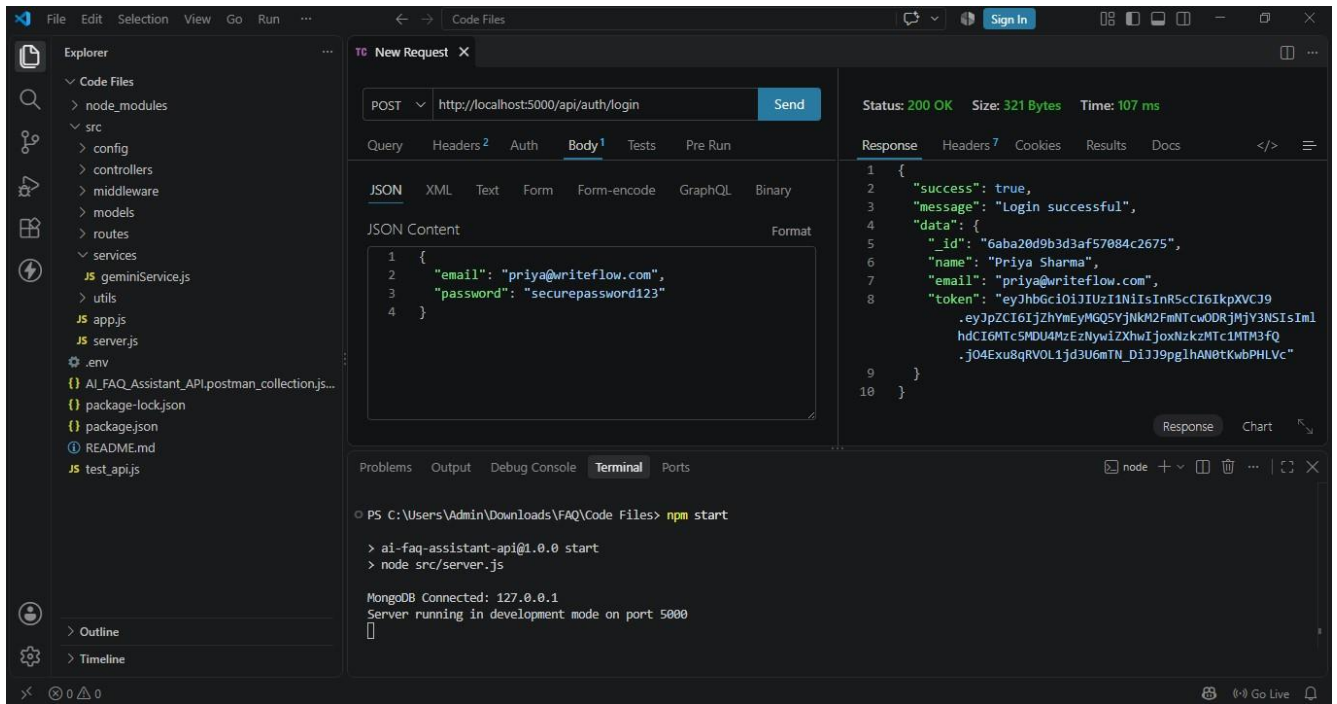
{

"email": "priya@writeflow.com",

"password": "securepassword123"

}

```

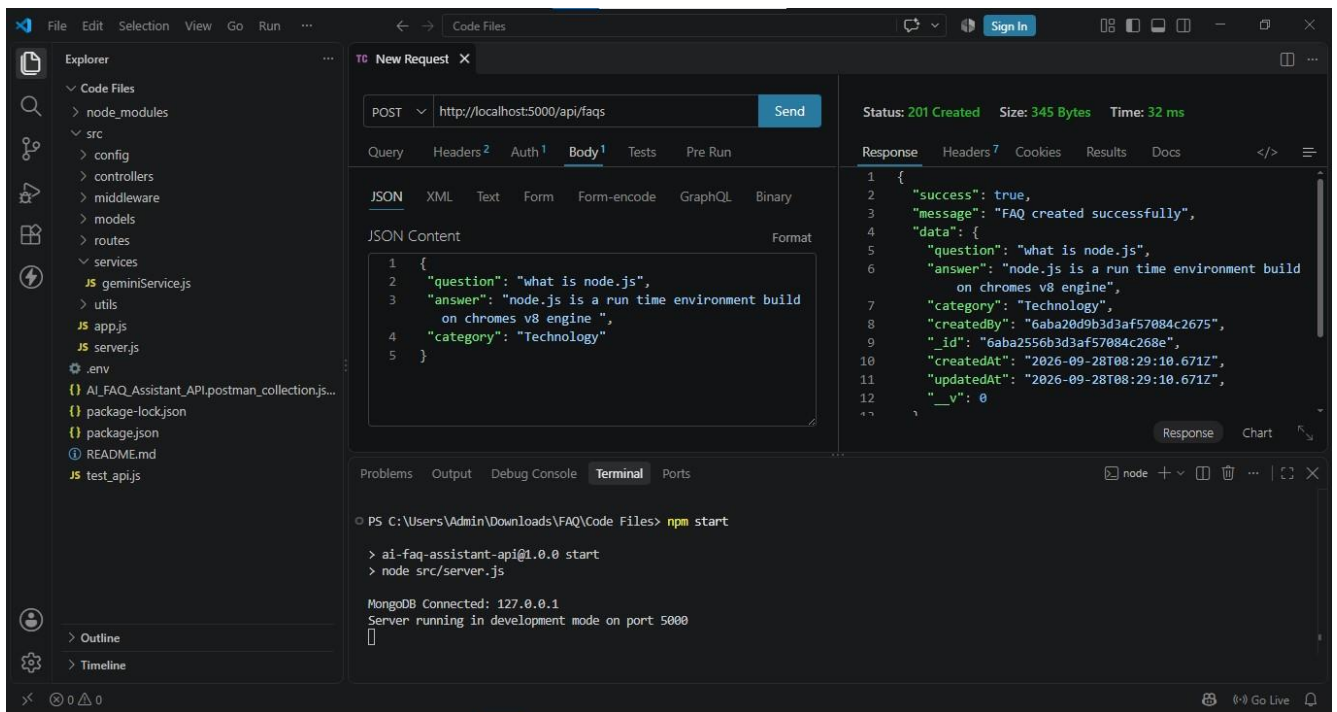


3. Resource Entity Creation (FAQ)

- Route: POST /api/faqs
- Access: Private (Bearer <JWT Token>)
- Payload Request Body JSON:

JSON

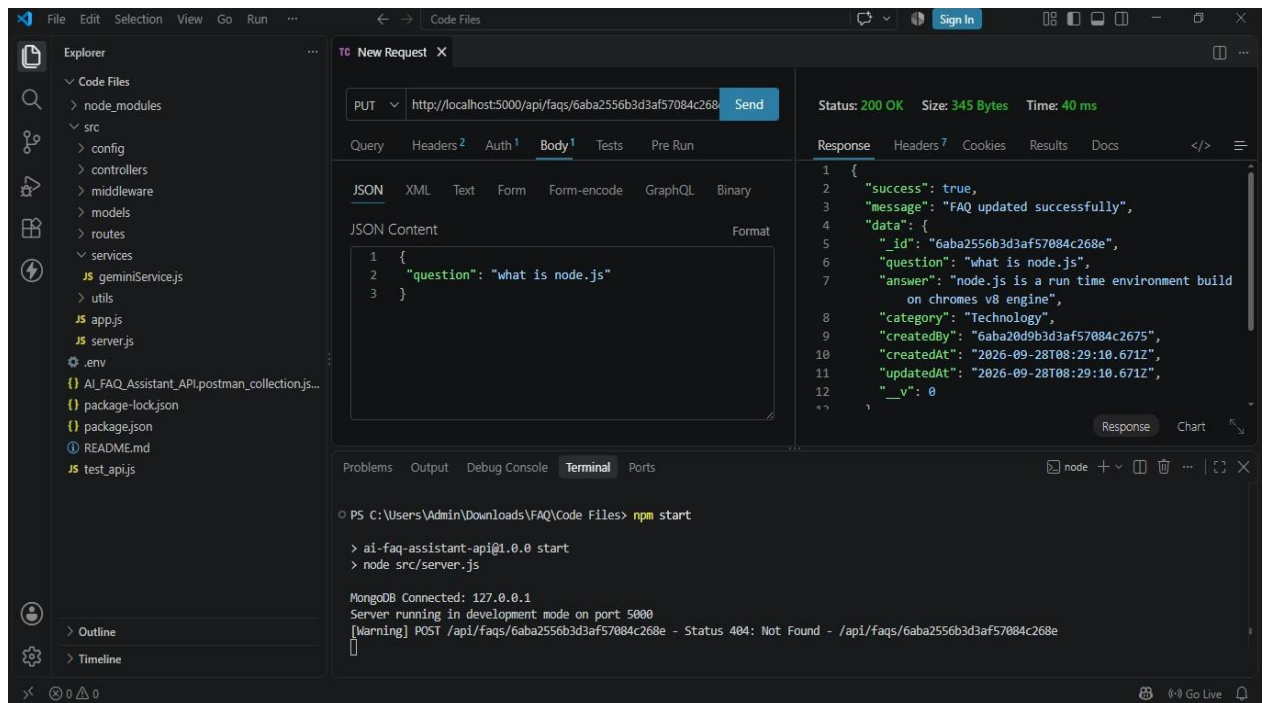
```
{  
  
  "question": "what is node.js",  
  
  "answer": "node.js is a run time environment build on chromes v8 engine.",  
  
  "category": "Technology"  
}
```



4. Keyword Resource Entity Fetching (Search)

- Route: PUT `/api/faqs/search?q=technology`

Access: Public



5. AI Intelligent Content Automation (Generate FAQ Pair)

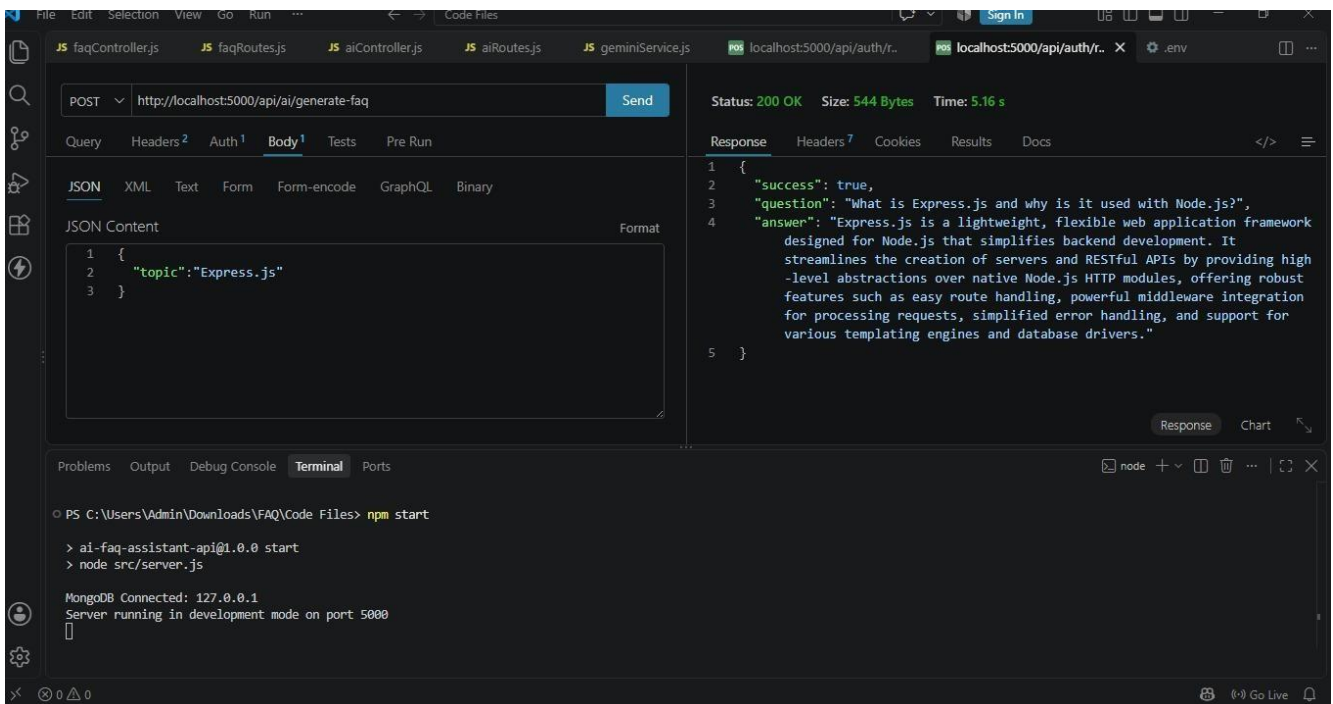
- Route: POST /api/ai/generate-faq •

Access: Private (Bearer <JWT Token>)

- Payload Request Body JSON:

JSON

```
{  
  
  "topic": "Express.js"  
}
```



14. SAMPLE WORKFLOW

- User registers through the authentication API.

- User logs in and receives an authentication token.
- Content creator sends a topic to the AI generation endpoint.
- The generated FAQ draft is reviewed and submitted to the FAQ creation endpoint.
- The FAQ is validated and stored in MongoDB with creator reference and category.
- Public users search and retrieve published FAQ content.
- Passwords are hashed using bcrypt before storage.
- JWT authentication protects restricted API routes.
- Authorization checks restrict creators to their own records, with broader permissions for administrators.
- Mongoose schema validation checks required fields and data structure.
- Centralized error handling standardizes errors and reduces exposure of internal database details.

15. LIMITATIONS AND FUTURE ENHANCEMENTS

- Add a dedicated frontend dashboard for administrators, creators, and FAQ consumers.
- Introduce analytics for frequently searched questions and unanswered queries.
- Add multilingual FAQ generation and search.
- Provide human review and approval before publishing AI-generated answers.
- Add pagination, rate limiting, audit logs, and monitoring as needed for deployment.
- Evaluate semantic/vector search if deeper meaning-based retrieval is required; the supplied reference currently describes keyword-based search.

16. CONCLUSION

AnswerBot provides a structured backend approach to FAQ management and customer support automation. By combining REST APIs, MongoDB, authentication, category organization, search, and AI-assisted FAQ generation, the system aims to reduce repetitive manual work and make support information easier to maintain and retrieve. The project can be extended with a frontend, analytics, and additional search capabilities.

17. PROJECT DEMO VIDEO

Demolink:

https://drive.google.com/file/d/1lAytiD9crM6pm25euHHNG3JRg2Yv_dGj/view?usp=drivesdk